08 October

Exam #1

EE 641: Fall 2025

Name: _____ USCid: _____

This content is protected and copyright reserved. It may not be shared, uploaded, or distributed.

Instructions: Print your name above. You may use a scientific calculator and one 8.5"x11" sheet for notes (front and back OK). You have 75 minutes to complete this exam. The exam consists of 13 multiple choice questions and 2 free-response questions — total: 78 points.

Multiple-choice questions Each multiple-choice question is worth 3 points. Select the best answer. When multiple options seem correct, choose the single best one.

1. Why does ShuffleNet require channel shuffling between group convolution layers?
   A. Without shuffling, each group influences only the same group in the next layer, blocking cross-group information flow.
   B. Shuffling prevents gradient vanishing by ensuring groups exchange information across channels during backpropagation.
   C. Group convolutions reduce FLOPs by $1/g$ but increase memory fragmentation; shuffling balances memory allocation across groups.
   D. Without shuffling, gradient contributions from different groups have unequal magnitudes, causing imbalanced feature learning.

2. In SSD, hard negative mining selects the highest-loss negative anchors rather than using all negatives. What does this address?
   A. Easy negatives drive loss toward zero prematurely, triggering learning rate decay before the model learns positive examples.
   B. Including 10K negatives per batch increases backward pass memory by $\sim$8GB, exceeding typical GPU capacity.
   C. Training on $\sim$10,000 negatives causes overfitting to background patterns, reducing the network's discrimination of object features.
   D. Easy negatives contribute small individual gradients that sum to dominate the total gradient, washing out the signal from positives.

3. For pose estimation, spatial heatmaps generally outperform direct (x, y) coordinate regression. What is the key structural difference?

   A. Regression gradient variance scales as $\sigma^2 \propto (H \cdot W)^2$, while heatmap variance is bounded by the spatial resolution.

   B. Heatmaps preserve spatial equivariance through convolutional processing, while regression flattens features into fully-connected layers that treat locations as independent outputs.

   C. Heatmaps represent multi-modal distributions over keypoint locations, while regression outputs a single (x, y) prediction.

   D. Heatmaps use softmax normalization across $H \times W$ locations for probabilistic outputs, while regression outputs raw (x, y) coordinates.

4. DenseNet concatenates layer outputs while ResNet adds them. What is the key difference in how features propagate?

   A. Concatenation grows feature maps by $+k$ channels per layer, while addition maintains constant C channels through summation.

   B. Addition enforces $\mathbf{x} + \mathbf{F}(\mathbf{x})$ alignment through element-wise sums, stabilizing gradients, while concatenation $[\mathbf{x}; \mathbf{F}(\mathbf{x})]$ creates channel dependencies.

   C. DenseNet reuses features without recomputation (forward cache), while ResNet recomputes activations at each block boundary.

   D. Concatenation creates separate gradient paths for each layer via channel slices, while addition merges layers into a single shared gradient path.

5. GAN mode collapse (generator producing limited variety) is linked to the reverse KL divergence $D_{KL}(p_g||p_{data})$. Why does this divergence favor mode-seeking behavior?

   A. Reverse KL minimizes $H(p_g)$, producing sharper distributions with lower entropy that concentrate on fewer modes.

   B. Generator capacity C limits simultaneous mode representation; reverse KL selects modes with $p_{data}(x) > 1/C$.

   C. Reverse KL weighs errors by p_g rather than p_{data}, prioritizing regions where p_g has high mass.

   D. Reverse KL assigns infinite penalty when $p_g > 0$ but $p_{data} = 0$, pushing the generator toward high-density regions.

6. In BPTT, the gradient backpropagates through a product of temporal Jacobians: $\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_{t-1}} = \text{diag}(\tanh'(\mathbf{z}_t)) \cdot \mathbf{W}_{ss}$. Why does repeated multiplication of these Jacobians cause vanishing or exploding gradients?

   A. Jacobian products accumulate float32 roundoff errors $\epsilon \approx 10^{-7}$; after $T - k$ multiplications, ϵ^{T-k} dominates the gradient.

   B. Backpropagation through $\tanh'(z) \leq 0.25$ compounds saturation; effective learning rate decays as $(0.25)^{T-k}$ with sequence length.

   C. Spectral radius $\gamma = \|\mathbf{W}_{ss}\| \cdot \max|\tanh'|$ compounds as γ^{T-k} over $(T-k)$ timesteps, causing exponential decay or growth.

   D. Recursive state $\mathbf{s}_t = f(\mathbf{s}_{t-1})$ creates feedback where $\delta\mathbf{W}$ at $t = 0$ amplifies to $\|\delta\mathbf{s}_T\| \propto \|\mathbf{W}\|^T \|\delta\mathbf{W}\|$.

7. Why does LSTM maintain both a cell state c_t and a hidden state h_t instead of using a single state vector like vanilla RNN?

   A. Cell state uses additive updates with derivative $\frac{\partial c_t}{\partial c_{t-1}} = f_t$, preserving gradients when $f_t \approx 1$.

   B. Separating $c_t \in \mathbb{R}^d$ and $h_t \in \mathbb{R}^d$ doubles capacity to $2d$ parameters, learning richer representations than single-state RNNs.

   C. Hidden state applies $h_t = \tanh(c_t)$ nonlinearity for predictions; cell state remains linear ($c_t = f_t \odot c_{t-1} + \ldots$) to avoid saturation.

   D. Cell state c_t stores long-term dependencies (> 100 steps) while h_t captures short-term patterns (< 10 steps), enabling multi-timescale operation.

8. Why is initializing $\mathbf{W}_{ss}$ to be approximately orthogonal (or near-identity) beneficial for vanilla RNN training?

   A. Orthogonal $\mathbf{W}_{ss}$ has norm 1, making spectral radius $\gamma \approx \max|\tanh'|$, which prevents rapid gradient decay.

   B. Orthogonal matrices bound forward activations $\|\mathbf{s}_t\| \leq \|\mathbf{s}_0\|$, preventing explosion during the forward pass.

   C. Near-identity $\mathbf{W}_{ss} \approx \mathbf{I}$ biases toward $\mathbf{s}_t \approx \mathbf{s}_{t-1}$ (copy mechanism), providing inductive bias for sequences.

   D. Orthogonal $\mathbf{W}_{ss}$ decorrelates hidden units ($\langle h_i, h_j \rangle = 0$ for $i \neq j$), preventing redundant feature learning.

9. In truncated BPTT with window $k = 20$, the forward pass processes a full sequence of length $T = 200$, but backpropagation only unrolls k steps. What limitation does this create?

   A. Forward states $\mathbf{s}_t = f(\mathbf{s}_{t-1}, \mathbf{x}_t)$ propagate information only k steps, limiting memory horizon to k timesteps.

   B. Truncation biases the loss estimate: sequences with $T > k$ contribute $\mathbb{E}[L|t \in [T-k, T]]$ rather than full $\mathbb{E}[L]$.

   C. The network learns patterns with period $\leq k$ only; periodicities $p > k$ lack representation in the k-step truncated graph.

   D. Gradient graph is cut at k steps ($\frac{\partial L}{\partial \mathbf{s}_{t-k-1}} = 0$), blocking parameter updates from dependencies beyond k.

10. A vanilla RNN has gradient decay factor $\gamma = 0.85$ and processes sequences of length $T = 100$ with full BPTT (no truncation). What determines the longest dependency the model can learn?

    A. Hidden state capacity $d = 512$ encodes $\log_2(d) \approx 9$ bits; with $\gamma = 0.85$, effective memory stores ≈ 6 timesteps.

    B. Gradients decay as γ^{T-k}; when this falls below float32 precision ($\approx 10^{-6}$), gradients underflow, limiting dependencies to ≈ 85 steps.

    C. Vanilla RNN lacks forget gates; information decays as $\|\mathbf{s}_t - \mathbf{s}_0\| \propto \gamma^t$ until $t \approx 1/|\log \gamma| \approx 6$ steps.

    D. Optimizer momentum $\beta = 0.9$ accumulates noise with variance $\sigma^2/(1-\beta^2)$; beyond T_{eff}, noise exceeds signal.

Questions 11–13 refer to the following scenario. Read the entire scenario before answering.

Setup: A research team trains a DCGAN on 50,000 images of building facades spanning eight architectural styles: Georgian, Victorian, Art Deco, Modernist, Brutalist, Gothic Revival, Bauhaus, and Postmodern. Each style has 6,000–7,000 examples (roughly balanced dataset).

Architecture:

- Generator: 128-dim $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I}) \to$ transposed conv layers with batch norm and ReLU $\to$ tanh output (64×64 RGB)
- Discriminator: strided convolutions with LeakyReLU
- Loss: standard minimax

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{data}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))]$$

- Optimizer: Adam with $\beta_1 = 0.5$, $\beta_2 = 0.999$, $\alpha_G = \alpha_D = 0.0002$, batch size 128

Training trajectory: By iteration 10,000, training has collapsed. The discriminator hit 92–95% accuracy around iteration 5,000 and stayed there. Generator loss climbed steadily starting around iteration 3,000, plateauing near $\log(0.01) \approx -4.6$.

Output pathology: 97% of generated samples are Georgian facades with minor variations in window placement and color. The other seven styles never appear.

Diagnostics at iteration 10,000:

- Generator gradient norms: 10^{-5} to 10^{-6} (early training: 10^{-2} to 10^{-3})
- Discriminator gradient norms: 10^{-2} to 10^{-1} (unchanged)
- Discriminator outputs: $D(\mathbf{x}) \approx 0.98$–$0.99$ for real, $D(G(\mathbf{z})) \approx 0.01$–$0.02$ for fake
- Discriminator logits (pre-sigmoid): $+4$ to $+5$ for real, -4 to -5 for fake

11. Given the discriminator outputs $D(G(\mathbf{z})) \approx 0.01$–$0.02$ and generator gradient norms $\sim 10^{-6}$, what is the primary mechanism causing the generator's vanishing gradients?

    A. Batch normalization in the generator rescales gradients by $1/\sqrt{\text{Var}[G(z)]}$; with $D \approx 0$ for fake samples, variance collapses to $\sim 10^{-6}$.

    B. Sigmoid derivative $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \approx 0.02$ at logit ≈ -4 creates the bottleneck, multiplying through the chain rule.

    C. Discriminator outputs $D \approx 0.01$ push the generator loss $\log(1 - D) \approx \log(0.99) \approx -0.01$ near zero, providing negligible learning signal.

    D. The generator loss $\log(1 - D)$ has derivative $-1/(1 - D) \approx -1/0.98 \approx -1.02$, which is close to zero and causes gradient attenuation.

12. The observation that the generator exclusively produces Georgian-style facades (ignoring 7 other equally-represented styles) is most directly explained by which property of the GAN training objective?

    A. The discriminator's 92% accuracy indicates it has memorized the Georgian mode during early training, biasing subsequent generator updates toward this style.

    B. The minimax objective drives p_g toward the mode with highest p_{data} density; Georgian facades have higher average pixel variance ($\sigma^2 \approx 0.15$ vs 0.08 for other styles), attracting more generator mass.

    C. Generator capacity with 128-dim $\mathbf{z}$ limits representable modes to $\log_2(128) \approx 7$ styles; the eighth mode (Georgian) dominates due to initialization bias.

    D. The JS divergence contains a reverse KL term $KL(p_g||p_{data})$ that assigns infinite penalty where $p_g > 0$ but $p_{data} = 0$, but zero penalty when p_g ignores modes.

13. The team wants to restore gradient flow to the generator without changing the discriminator architecture or switching to a fundamentally different GAN variant. Which modification most directly addresses the saturated gradient problem?

    A. Reduce discriminator learning rate to $\alpha_D = 0.00004$ (factor of $5\times$), slowing discriminator convergence and preventing it from saturating the generator loss.

    B. Use non-saturating loss $\mathcal{L}_G = -\mathbb{E}_z[\log D(G(z))]$ with derivative $-1/D$, yielding gradient magnitude ≈ 50 at $D \approx 0.02$.

    C. Increase generator learning rate from $\alpha_G = 0.0002$ to 0.001, amplifying gradients by $5\times$ to compensate for the 10^{-6} gradient norms.

    D. Add gradient clipping with threshold $\|\nabla\| \leq 1.0$ to prevent discriminator gradients from dominating; this stabilizes the ratio between generator and discriminator updates.

Free-response questions

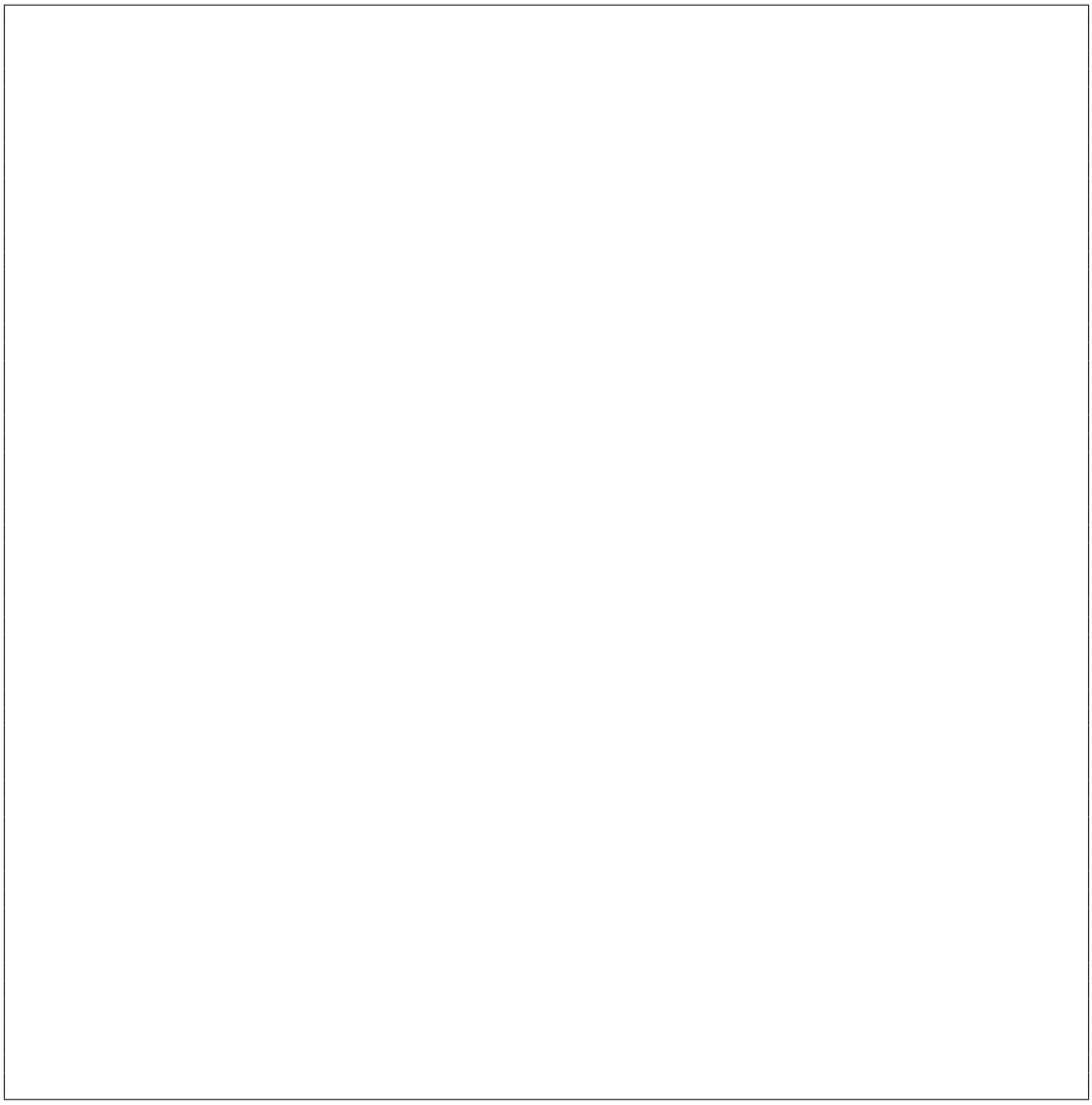
Show all work and box answers where appropriate.

14. **Multi-Scale Object Detection with Feature Pyramids**

Modern object detectors use feature pyramids to detect objects at multiple scales. Consider a single-stage detector with:

- Feature Pyramid: Creates features at scales $\{P_3, P_4, P_5\}$ with spatial resolutions $\{64 \times 64, 32 \times 32, 16 \times 16\}$ for a 512×512 input image
- Anchors: Each scale has anchors with base sizes $\{32, 64, 128\}$ pixels respectively
- Matching: Anchors with IoU > 0.5 are positive, IoU < 0.3 are negative, others are ignored

(a) [6 points] A 20×20 pixel object in the input image will occupy different amounts of space on different feature maps. Calculate how many spatial cells this object occupies on P_3 (64×64) versus P_5 (16×16). Then explain why detecting this small object on P_3 is more effective than on P_5.



(b) [8 points] Consider a ground truth object with bounding box coordinates (in pixels): $[x_1, y_1, x_2, y_2] = [100, 100, 140, 140]$ (a 40×40 box).

Three anchors are generated at feature map P_3:

- Anchor A: $[95, 95, 127, 127]$ (size 32×32, center close to GT)
- Anchor B: $[90, 90, 154, 154]$ (size 64×64, center close to GT)
- Anchor C: $[120, 120, 152, 152]$ (size 32×32, center offset from GT)

(i) Without computing exact IoU values, rank these anchors by expected IoU (highest to lowest) and briefly justify your ranking.

(ii) Given: Anchor A has IoU $= 0.55$, Anchor B has IoU $= 0.48$, and Anchor C has IoU $= 0.25$. Classify each anchor as positive, negative, or ignored. Why is having an "ignored" category important for training?

15. **VAE KL Divergence Derivation and Interpretation**

In a Variational Autoencoder, the encoder network outputs parameters $\boldsymbol{\mu}(x)$ and $\boldsymbol{\sigma}(x)$ that define the approximate posterior $q_\phi(z|x) = \mathcal{N}(\boldsymbol{\mu}(x), \text{diag}(\boldsymbol{\sigma}^2(x)))$ over a d-dimensional latent variable $\mathbf{z}$. The prior is a standard Gaussian $p(z) = \mathcal{N}(0, \mathbf{I})$.

(a) [12 points] Starting from the KL divergence definition:

$$D_{KL}(q||p) = \mathbb{E}_{\mathbf{z}\sim q}\left[\log \frac{q(\mathbf{z}|x)}{p(\mathbf{z})}\right]$$

Derive the closed-form expression for $D_{KL}(q_\phi(\mathbf{z}|x)||p(\mathbf{z}))$. Work through the following steps:

Step 1: Write the log ratio $\log q(\mathbf{z}|x) - \log p(\mathbf{z})$ using the Gaussian PDFs. The 2π terms should cancel.

Step 2: Simplify the expression to show it factors as a sum over dimensions:

$$\log \frac{q(\mathbf{z}|x)}{p(\mathbf{z})} = \sum_{j=1}^{d} [\text{expression in } z_j, \mu_j, \sigma_j]$$

Step 3: Take the expectation over $z_j \sim \mathcal{N}(\mu_j, \sigma_j^2)$ for each dimension. Use:

$$\mathbb{E}[z_j] = \mu_j, \quad \mathbb{E}[z_j^2] = \sigma_j^2 + \mu_j^2$$

You should arrive at:

$$D_{KL}(q||p) = \frac{1}{2} \sum_{j=1}^{d} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right)$$

(b) [5 points] Consider a VAE with a 2-dimensional latent space ($d = 2$). At a particular input x, the encoder outputs:

$$\boldsymbol{\mu}(x) = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad \boldsymbol{\sigma}(x) = \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix}$$

Calculate the exact KL divergence $D_{KL}(q_\phi(\mathbf{z}|x)||p(\mathbf{z}))$. Show your computation for each dimension.

(c) [8 points] Interpret the role of each term in the KL expression:

(i) What does the term μ_j^2 penalize, and why does this make sense given the prior $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$?

(ii) The term $\sigma_j^2 - \log \sigma_j^2 - 1$ penalizes deviation from what value of σ_j?

(work sheet, intentionally blank)